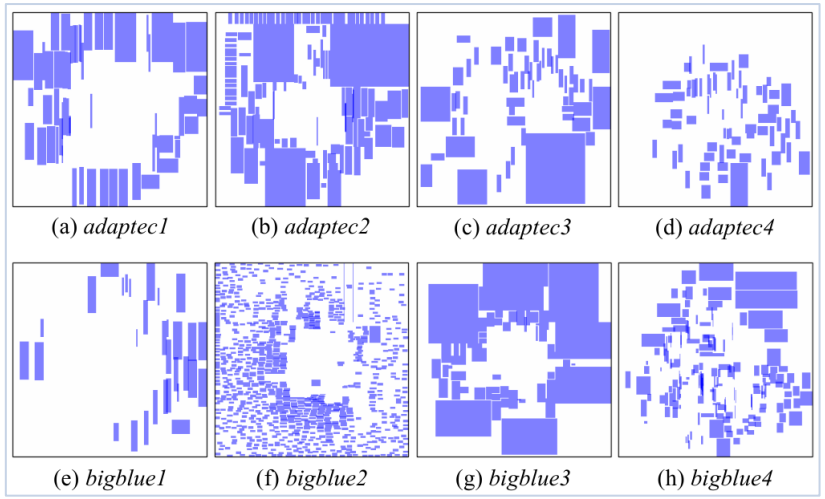
六篇共性：制造数据集

论文	任务方向	① 数据集从哪来
FlowPlace（DAC'26）	宏单元布局、生成式	合成 + 领域先验；合法位置二值掩码 M_i(p)，边界偏置 1/dist² 加权采样，再按空间邻近反推 pin 与连接
EIM（DAC'26）	宏单元布局、RL 奖励	专家数据；每设计 50 个人工专家布局，展开成长度=宏单元数的轨迹
MacroDiff（IJCAI'26）	宏单元布局、扩散	合成 + 图增强；保度数随机重连，每设计 1000 个共 8000；ground truth 用 DREAMPlace 生成
Accelerating Detailed Routing（Data'26）	详细布线、RL 调度	自造轨迹数据；17 设计 5782 次完整布线，60% 扰动采样 + 40% Sobol 低差异采样
DeepLayout（ICML'25）	布局表征、自监督	绕开标注；RISCY 四个设计 6000+ 无标注布局，直接自监督
QiMeng-SALV（NeurIPS'25）	RTL 生成、LLM	爬取 + 清洗 + 功能校验；CodeV 165k → 清洗 135k；Yosys 逐信号验证产出 110k 部分正确偏好对

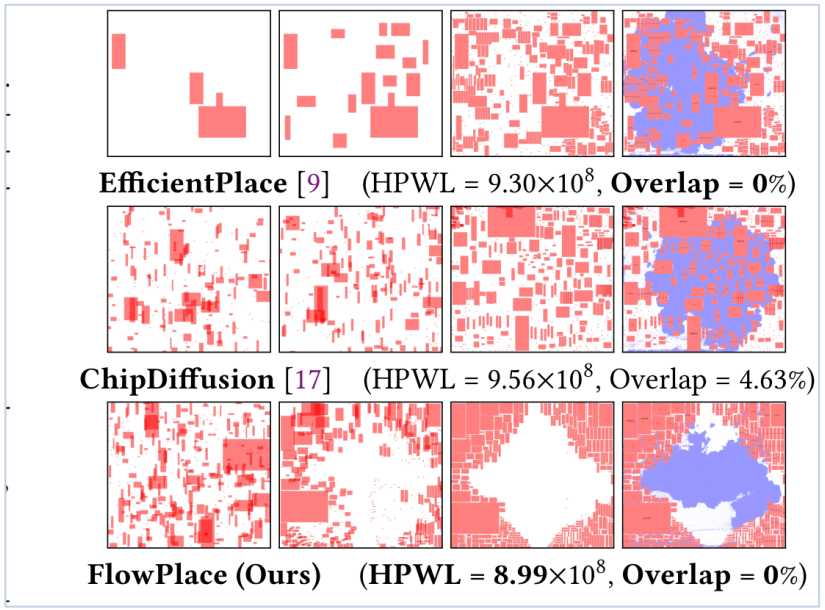
共性：AI4EDA 没有公开工业数据集

前五篇为AI驱动的物理设计，最后一篇是AI生成RTL代码，不涉及物理设计

六种造数据的策略



MacroDiff+ 在 ISPD2005 MMS 八个设计上的生成结果（合成+图增强）



FlowPlace 在 superblue7 的布局对比（合成+领域先验）

① FlowPlace：合成 + 领域先验

网格化 + 合法位置掩码 + 边界偏置打分

② EIM：专家标注

每设计 50 个人工专家布局（真实但不可规模化）

③ MacroDiff+：合成 + 图增强

保度数重连，ground truth 用 DREAMPlace

④ Accelerating Detailed Routing：轨迹自采样

5782 次完整布线，扰动 + Sobol，无需人工

⑤ DeepLayout：自监督无标注

直接用原始布局，回避标注成本

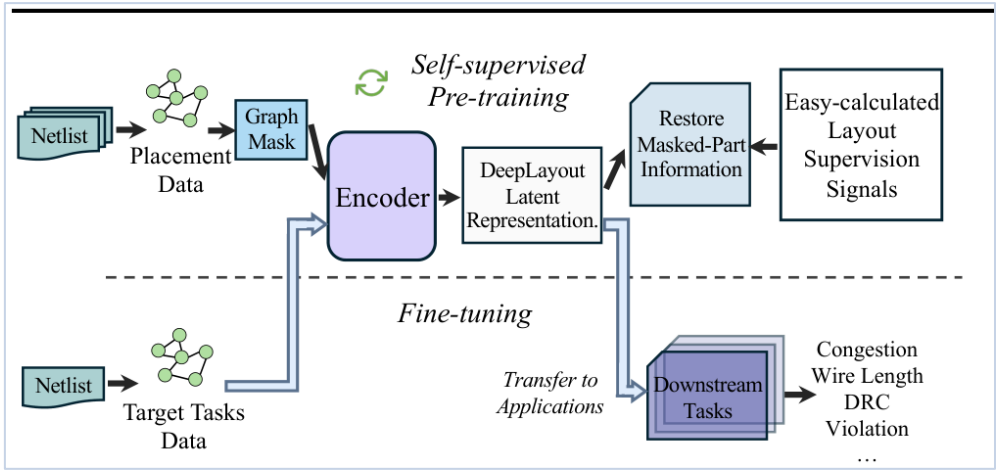
⑥ SALV：爬取 + 功能校验

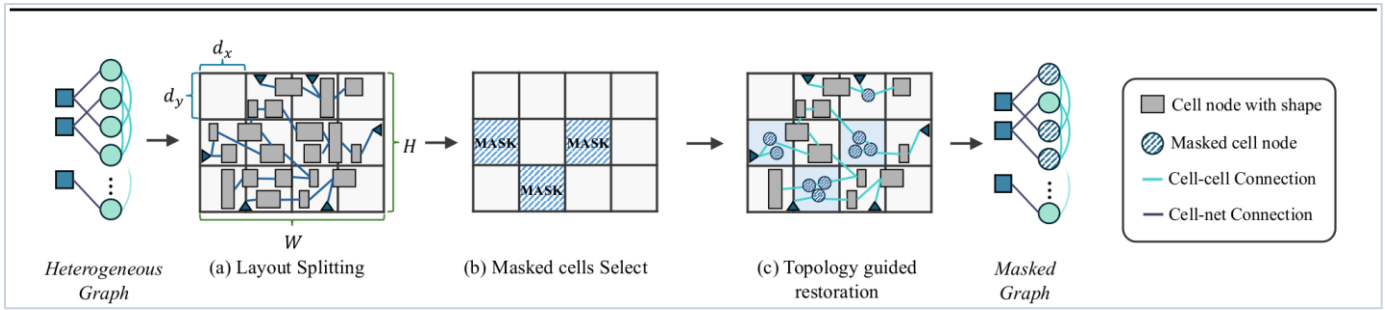
GitHub 爬 Verilog，Yosys 逐信号判正确性

1、 DeepLayout（ICML'25）

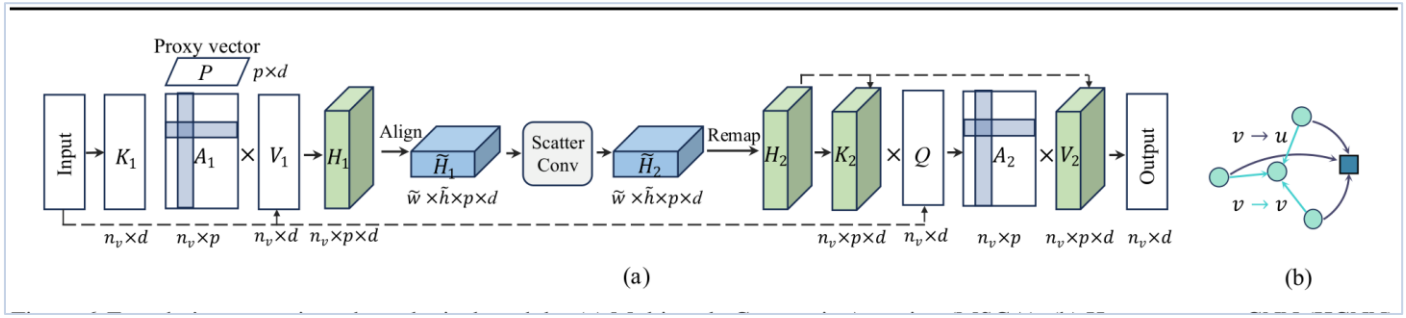
后端第一个通用表征框架：

自监督预训练 + 多任务迁移





面向布局的掩码策略流程



编码器：(a) MSGA　(b) HGNN

明确划清前后端的界

前端表征不适用于后端：逻辑功能和物理性能不强相关，物理设计和物理布局相关

三个针对后端的 mask 改动

网格化切分而非固定 patch，只从非空网格选 mask 对象

mask 掉的 cell 坐标置 0 但保留尺寸和属性

保留被 mask cell 的拓扑连接边（类比 ECO 增量修改）

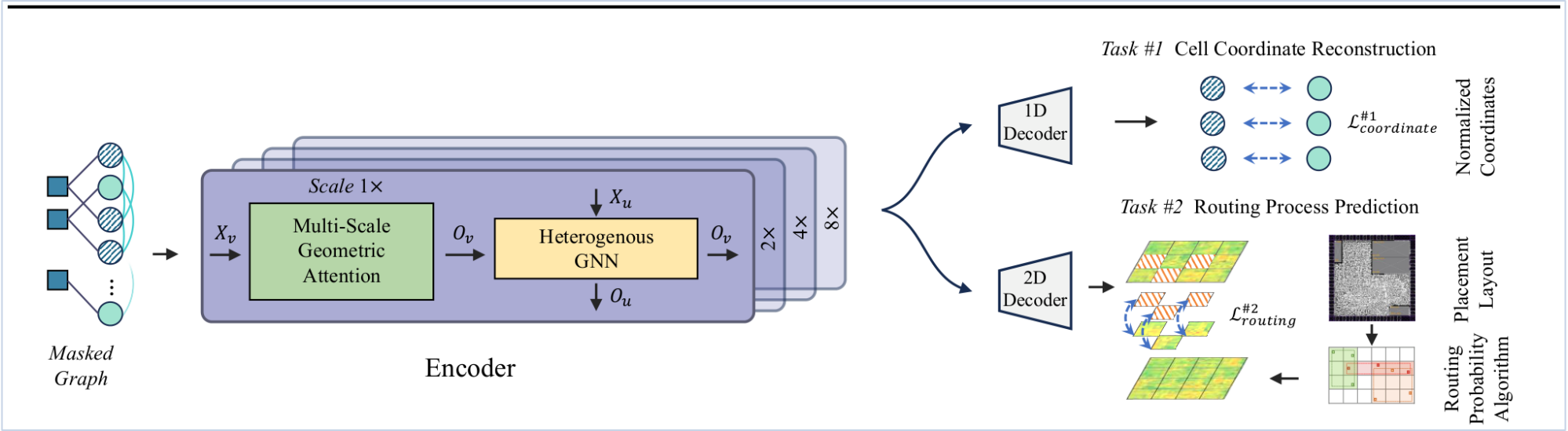
为什么能绕开标注

原始无标注电路数据是可获得的：开源仓库、中间参数配置、历史设计数据库

自监督学习在图像和文本上已成熟

只用 5 个样本微调，就超过了用 20 个样本的专用模型

1、DeepLayout 预训练与结果



拥塞预测（5 样本设定）	Pearson	MAE	RMSE	SSIM
GPDL	0.2679	0.2418	0.2476	0.2926
CircuitGNN	0.2978	0.0166	0.0520	0.7395
CircuitPoint	0.2783	0.1061	0.1148	0.0943
DeepLayout（本文）	0.4270	0.0146	0.0379	0.7718

省流：

1、只验证了评估类任务，没有验证能不能用来做生成或优化，对实际工作的帮助有限；

2、此前很少有人将AI引入物理设计，这是第一篇，具有开创意义

2、FlowPlace（DAC'26）

FlowPlace 总体流程（自绘，据 DAC'26 正文整理）

① 掩码引导合成数据

- 画布网格化 + 合法位置二值掩码 M_i(p)
  宏单元按尺寸降序，GPU 并行避免串行拒绝采样
- 边界偏置打分 1/(dist+ε)² 加权采样
  依据：工程师习惯把大宏单元靠边界放
- 按空间邻近生成 pin offset 与连接
  重建 netlist → 得到成规模的合成布局

② 流匹配替代扩散

- GATv2 感知局部连接 + 多头自注意力捕捉全局交互
  与 ChipDiffusion 保持一致以保证公平对比
- 学习确定性速度场 v_theta(x_t, t, G)
  训练直接回归常值速度 x1 − x0，不需调噪声调度
- 推理用 Euler 解 ODE，采样 20–50 步
  源分布可自由选择（用画布上的均匀分布）

③ 硬约束引导采样

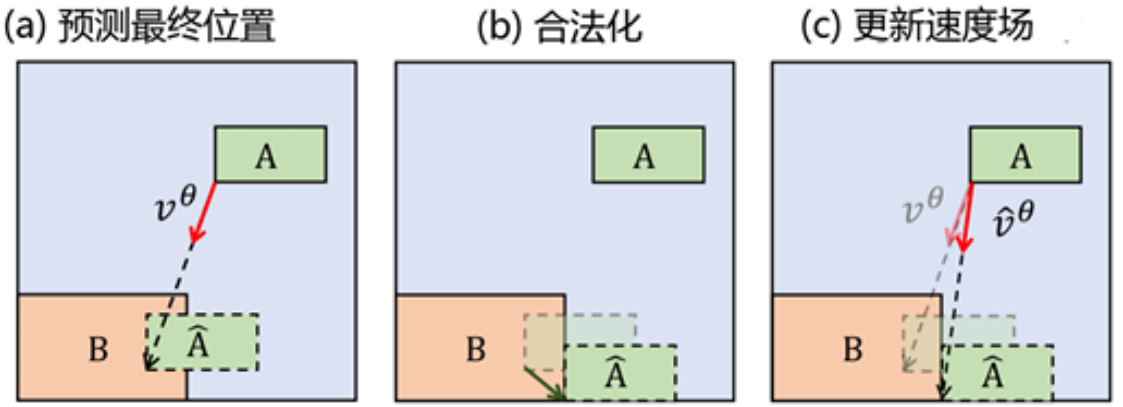
- 外推预测终点 x1 = x_t + (1−t)·v_theta
  投影算子 C(·) 做贪心合法化 → x1'
- GPU 并行几何检查；选格点时兼顾预测位置
  与到边界的距离，以最小化调整量
- 重算修正速度 v_theta' = (x1' − x_t)/(1−t)
  插值回轨迹 → 最终保证零重叠

三个改动同时成立且各有消融支撑：造数据 → 换生成范式 → 换硬约束处理

省流：

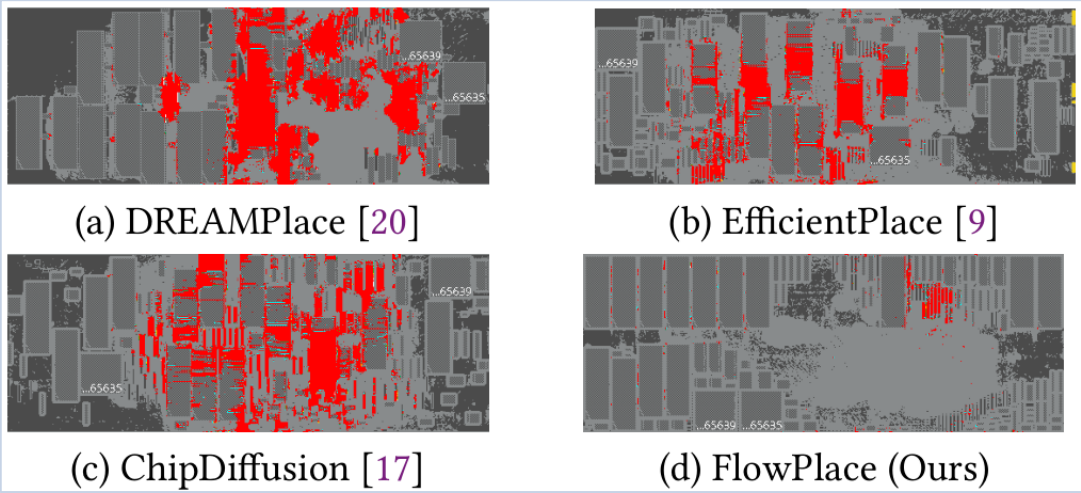
1、数据生成：将网表划分为均匀网格单元，计算位置的二值掩码并打分，再根据靠近边界的程度二次赋分（越靠近边界分数越高），按与分数成正比的概率从合法位置中采样宏单元的位置

2、利用流匹配模型，给每个宏单元设置坐标信息，并根据模型给宏单元一个速度值，经过匹配并移动后，更细新所有单元的速度，直到完成所有宏单元的布局



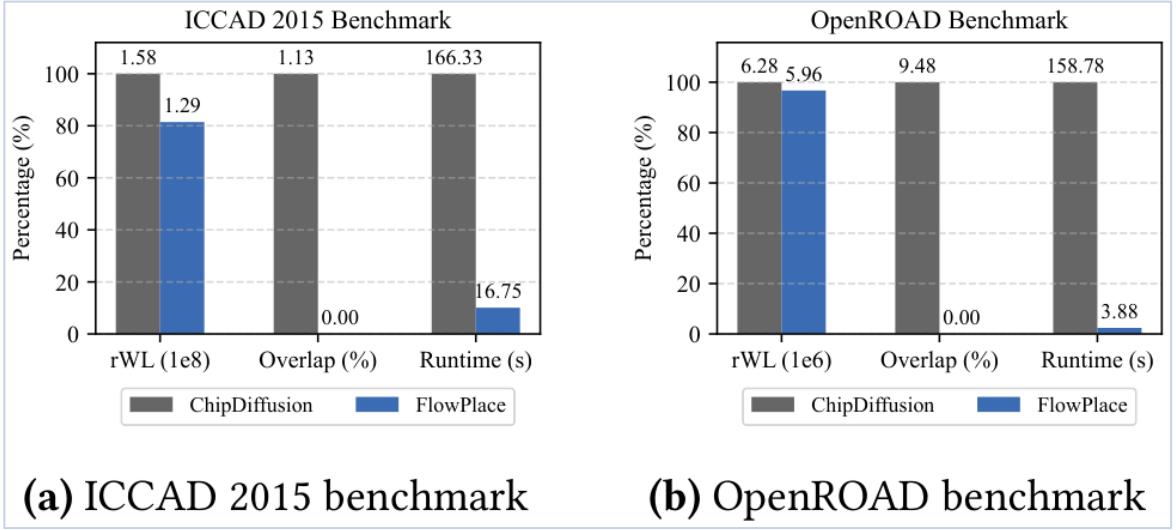
2、FlowPlace 数据集与结果

ICCAD2015 平均排名 rWL 1.62 / rO-H 1.00 / rO-V 1.25



(a) DREAMPlace [20]　(b) EfficientPlace [9]

(c) ChipDiffusion [17]　(d) FlowPlace (Ours)



(a) ICCAD 2015 benchmark　**(b)** OpenROAD benchmark

不知道写点啥，**Minimax**如是说：

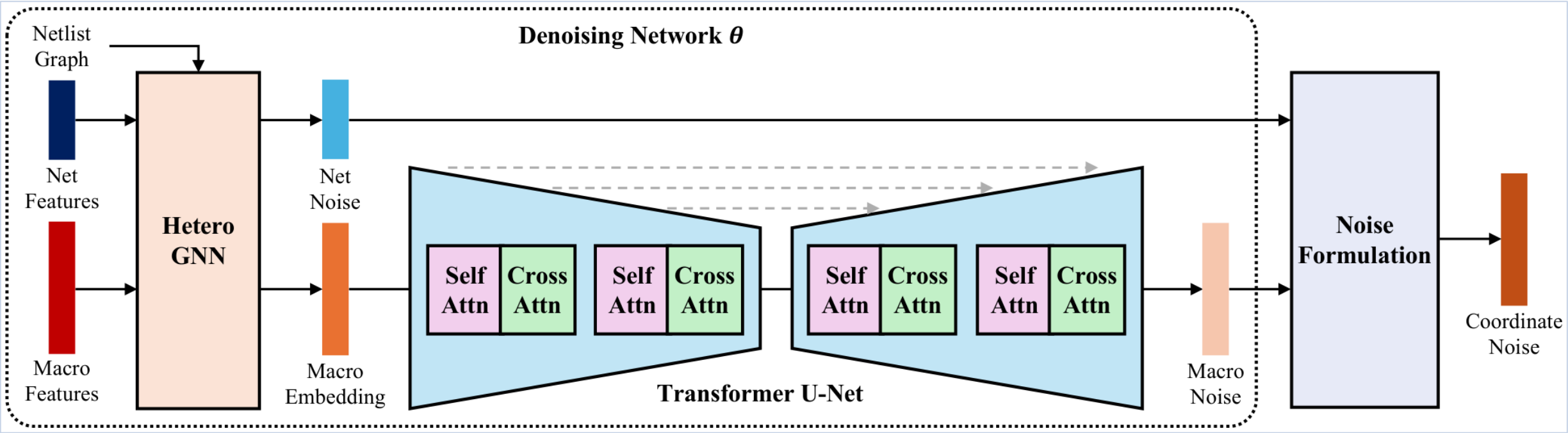
左图：定性 —— 红色是拥塞热点

- 红色区域 = 布线拥塞热点，越少越好
- (a)(b)(c) 三个基线均有成片红色，(d) FlowPlace 几乎没有
- 同尺寸宏单元成组平铺、空白分布规整
- 这是右图 Overlap = 0.00% 的直观对应

右图读法与全流程结果

- 纵轴是相对值（ChipDiffusion 归一到 100%），柱顶为绝对值
- rWL：ICCAD 81.6%、OpenROAD 94.9%
- Runtime 快 9.9×（ICCAD）/ 41×（OpenROAD）
- OpenROAD 全流程 WNS/TNS/Power/Area 均最优

3、MacroDiff+（IJCAI'26)



省流：

1、与上一篇论文相对应，这一篇解决的是同一问题，但上一篇用的是流匹配模型，这一篇用的是扩散模型

2、解决主要问题：现有扩散方法把布局纯粹当成几何图案生成，完全没有显式建模 netlist 拓扑，而拓扑才是决定线长的根本

3、本文方法：把拓扑和几何分开建模，用Hetero GNN 分支建模拓扑结构，利用 Transformer 分支建模几何结构

3、MacroDiff+ 数据集生成

环节	做法
增强方法	度保持配置模型
增强规则	保留所有 macro 节点和 net 节点的度数，只把边随机重连
规模	每个设计生成 **1,000** 个增强 netlist × 8 个设计 = **8,000** 个样本
标签（**ground truth**）	对每张增强 netlist 跑一遍 **DREAMPlace**，得到高质量参考布局作为 x_o

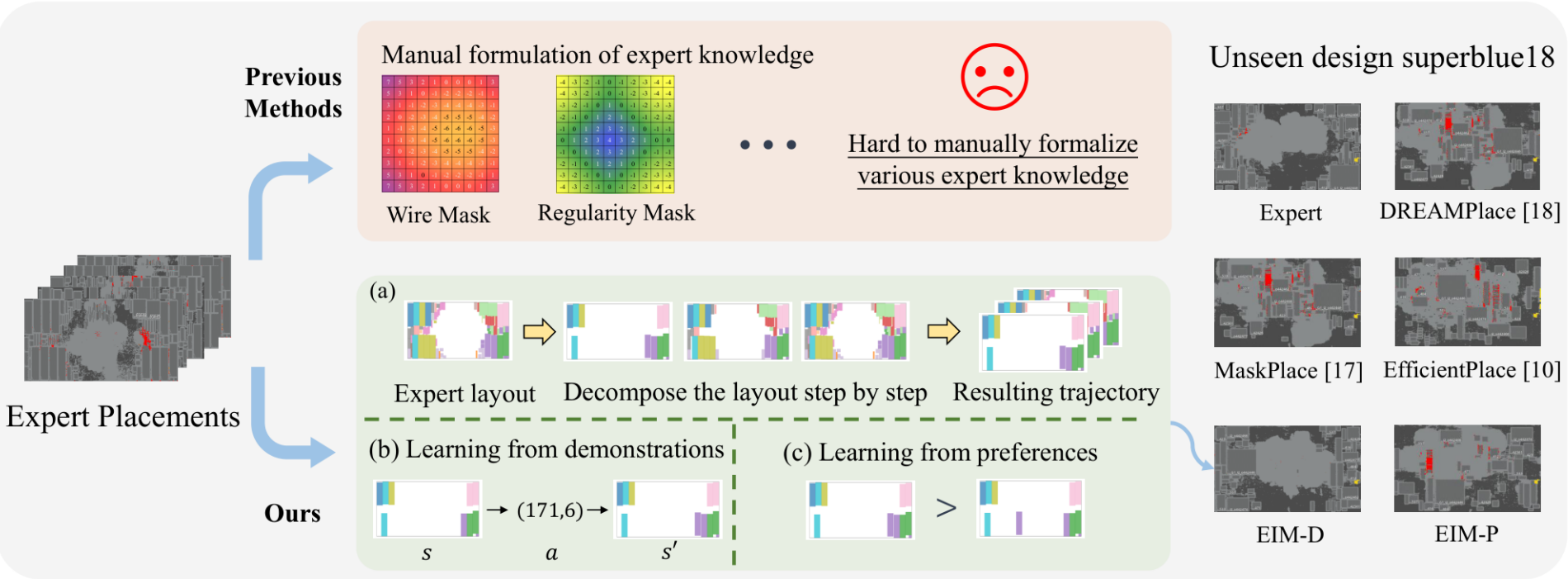
省流：

1、度保持配置模型模型：宏单元数、pin 数、net 数全部不变，但能生成几位大量的数据集

2、可能存在问题：生成的数据集网表有可能从结构上不正确，原文没提到这一块

4、EIM（DAC

从专家最终布局反推奖励模型



绕开形式化

专家知识难以写成公式，但专家的最终布局 readily available 。于是不写奖励公式，直接从结果反推。

诊断：RL 布局达不到专家水平，主因是奖励设计。HPWL 只反映线长，可布线性 / halo / 供电 / 电气特性如何映射到 PPA 仍是开放问题。

EIM-D（从展示学习）：把一张最终布局展开成长度=宏单元数的轨迹，每步产出 (s, a, s′)，用 IQ-Learn 做逆强化学习。

EIM-P（从偏好学习）：诱导偏好对 (s, a_chosen, a_random)，用 Bradley-Terry 损失 + $\alpha|r|^2$ 正则训练奖励模型。

5、EIM 数据集与结果

专家数据：每设计 50 个人工专家布局

设计（reward accuracy）	随机	MaskPlace	EIM-D	EIM-P
ariane133（训练）	0.02	0.54	0.77	0.99
superblue1（训练）	0.04	0.12	0.78	1.00
ariane136（未见，相似）	0.07	0.45	0.64	0.99
superblue18（未见，差异大）	0.09	0.14	0.38	0.09

反直觉发现

EIM-P 训练集准确率几乎满分，一到差异大的未见设计掉到 0.09，和随机模型没区别——严重过拟合。EIM-D 训练集落后，泛化却明显更好，最终 PPA 也更好。

领域地位

RL 布局从 2021 年 AlphaChip 以来，第一次认真处理「奖励天花板」问题

方法论上是范式迁移：把 RLHF / 模仿学习那套搬进 EDA，是 2023–2026 年 LLM 领域思路反向流入 EDA 的典型案例

和 FlowPlace 同组同门（南大 LAMDA + 华为诺亚）：一条走生成式、一条走奖励学习，合起来是该团队对这个方向的完整布局

奖励准确率高 ≠ 最终 PPA 好。判断奖励模型必须看泛化集上的准确率，而不是训练集拟合程度。

PPA：EIM-D 相比两个 RL 基线 rWL 降 8.23%、拥塞降 43.35%；相比 DREAMPlace rWL 降 14.98%、拥塞降 92.75%。

软肋：极度依赖专家数据，一个设计 50 个人工布局，作者自己把「用合成数据增强」列为 future work。

5、Accelerating Detailed Routing　详细布线

RL 调度参数，而不是 RL 替代算法

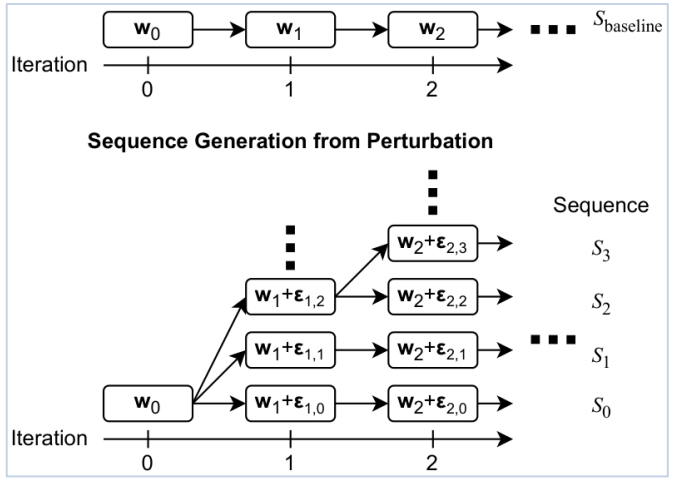


Fig.1　基线单一权重序列 vs 扰动采样生成多条序列

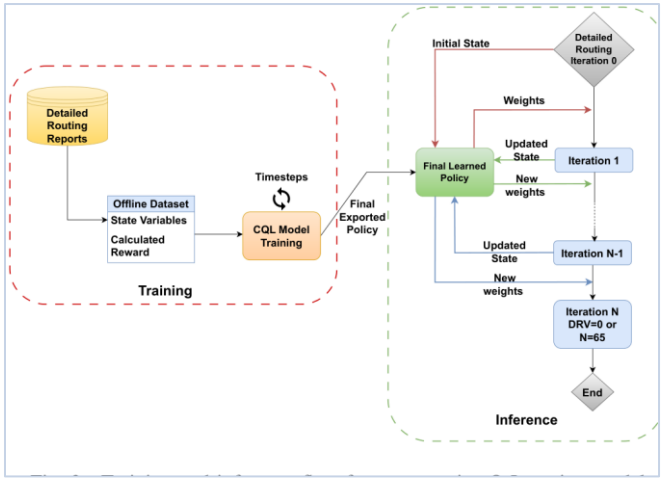


Fig.3　CQL 训练与推理流程

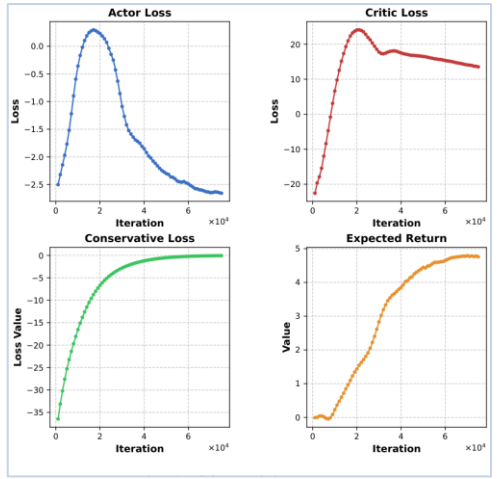


Fig.4　CQL 训练指标（actor / critic 均先过冲再回落）

观察：权重是静态且手工的

Dr.CU 的 violation cost 全程不变；OpenROAD 只按当前 ripup 迭代号调整

都不随设计变化，且都是领域专家手工设定

做法：CQL 动态选权重

每轮选 drcCost / markerCost / fixedShapeCost / markerDecay 四个权重

选 CQL：episode 要几分钟到几小时，online 探索不现实；保守惩罚把策略约束在数据支撑集内；推理只需一次前向

工程：插进现有工具

d3rlpy 训练 → TorchScript + LibTorch 嵌入 OpenROAD 的 C++ 路由器

每轮迭代结束抽状态、推理下一轮权重，开销约 2 秒（已计入所有 runtime）

5、Accelerating Detailed Routing 结果

学出来的权重趋势与领域专家的「常识」差异很大

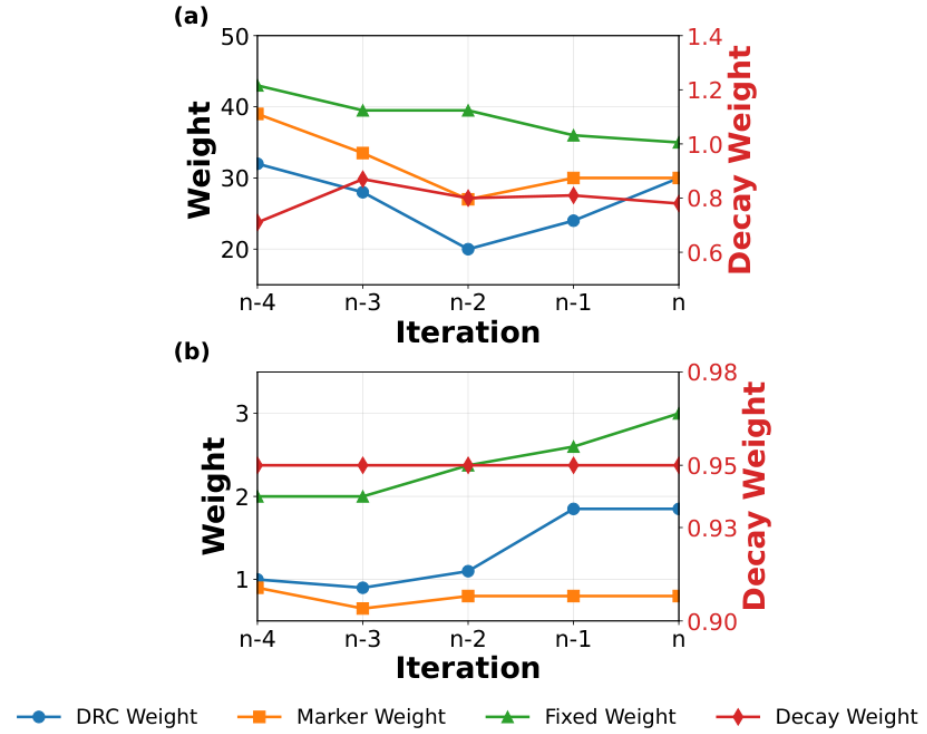


Fig.2 (a) 最优 vs (b) 基线权重趋势，最后几轮差异明显

最重要的发现

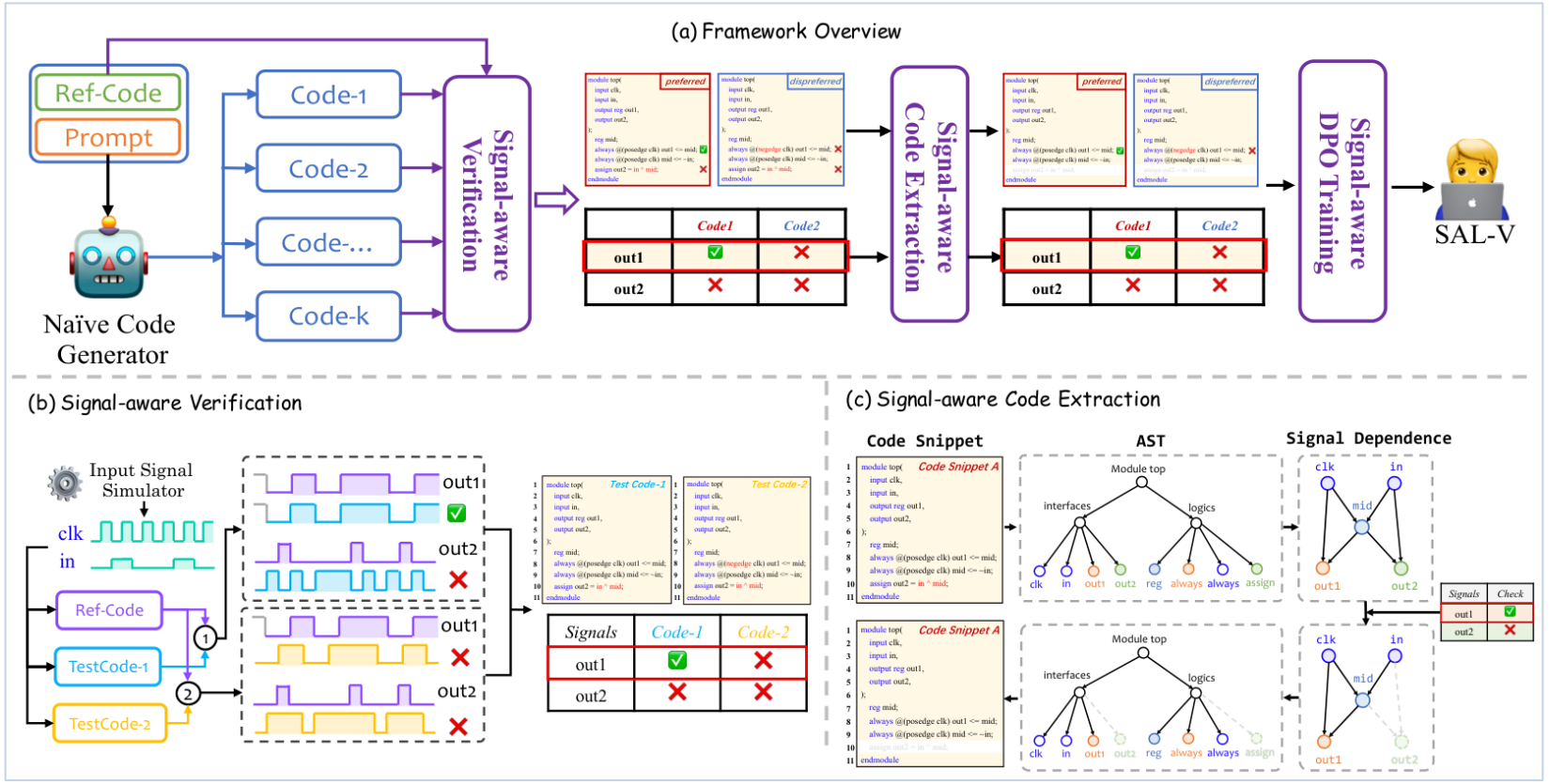
学出来的权重趋势跟基线专家设定的差异很大 —— 说明现有手工权重调度远非最优。作者的解释是代价函数之间是相对关系，可能存在多条等效的权重轨迹。

ISPD19 未见集：迭代数平均少 5%、最多 31%；runtime 平均快 1.56×、最多 3.01×；总 runtime 8098 s → 5200 s（35.79%）。

⚠ 负结果：小设计上推理开销占主导 —— gcd 3 s→4 s、ibex 27 s→28 s、ispd19_test9 758 s→772 s。基线本来就快速收敛，2 秒固定开销吃掉收益。

领域地位：AI 调度成熟算法而非替代算法，插入现有工具、不改算法主体，工业接受度最高。

6、QiMeng-SALV (NeurIPS'25)



用 Verilog 的信号并行性破局

恶性循环：高质量 SFT 数据稀缺 → 模型初始能力不足 → 难以采样到功能正确的模块 → 无法提供有意义的功能奖励 → RL 优化受限。

破局点：Verilog 各输出信号相对独立、并行产生 —— 即使模块整体错误，其中仍可能存在功能正确的信号实现。既有 RL 方法完全浪费了这个信息。

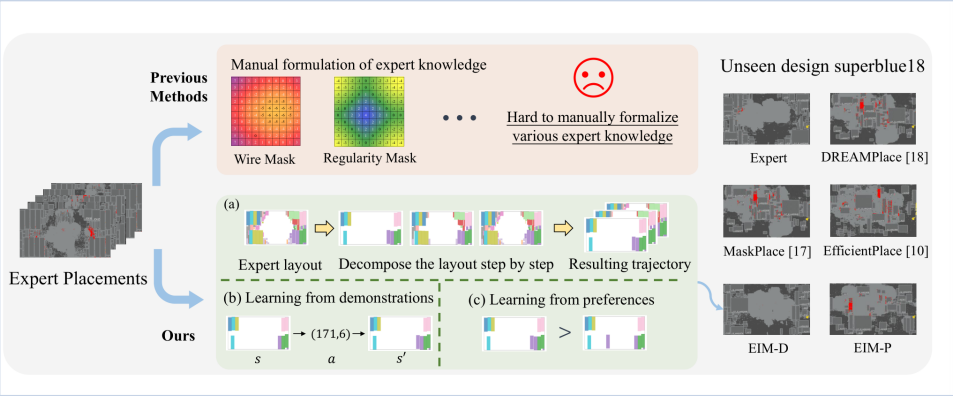
① Verification：Yosys 生成随机输入激励，逐信号对比生成模块与参考模块的输出

② Code Extraction：基于 AST 构建信号依赖拓扑图，反向遍历提取目标信号的代码段

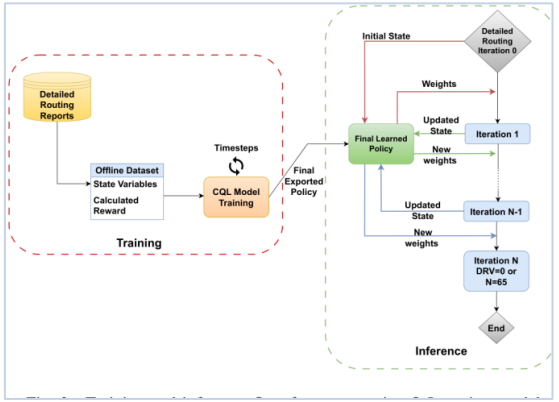
③ DPO Training：仅在对比信号相关代码段上计算 token 概率损失，过滤掉错误信号带来的噪声

对比： RL 的两种用法：EIM vs Accelerating Detailed Routing

一个替代手工奖励，一个只调度内部参数



EIM Fig.1 学奖励：绕开形式化，从专家最终布局反推



Accelerating Detailed Routing Fig.3 学调度：插进 OpenROAD 路由器，只选 violation cost 权重

维度	EIM（宏单元布局）	Accelerating Detailed Routing（详细布线）
RL 位置	替代手工奖励，策略自己学	不替代算法，只调度内部参数
插入点	MaskPlace 策略网络	OpenROAD A* 路由器的 violation cost
数据	专家布局（每设计 50 个）	自采样轨迹（17 设计 5782 次布线）
算法	IQ-Learn / Bradley-Terry	CQL（保守、离线）
落地形态	替换布局器内核	插进现有工具，约 2 秒开销
收益	rWL 降 14.98%（对比 DREAMPlace）	runtime 1.56×、迭代数少 5%
风险	依赖专家数据；EIM-P 泛化崩溃	小设计上负收益；复杂度收益比可质疑

共同点：都在解决「专家知识 / 超参如何形式化」—— 一个从结果学（逆向），一个从数据学。Accelerating Detailed Routing 更可能被工业界接受，因为它不改算法主体。

总结

三条可复用的经验、两个必须警惕的结论、三个可切入的方向

① 三条可复用的经验

数据集构建是 AI4EDA 论文的一号创新点，六篇无一例外，而造法与模型形态是强绑定的

泛化能力已成一级指标：四种解法——预训练+零样本推理、奖励模型迁移、表征预训练+少样本微调、跨设计跨工艺数据迁移

效率被当成一等目标：流匹配快 10–50×、MacroDiff+ 在 bigblue2 上快 235×、DeepLayout 5 样本超 20 样本专用模型

② 两个必须警惕的结论

奖励准确率高 ≠ 最终 PPA 好。EIM-P 训练集准确率 0.99，一到差异大的未见设计掉到 0.09，和随机没区别

学出来的调度 ≠ 领域专家的「常识」。Accelerating Detailed Routing 学到的权重趋势跟手工设定差异很大，说明现有调度远非最优

两篇同期论文在硬约束上判断相反，且 benchmark 不同 —— 文献里这块是空白

③ 值得切入的方向（P0）

合成数据 + 专家数据混合：FlowPlace 走合成（量大、先验人造）、EIM 走专家（真实、量小），两篇同期同门却没有交集 —— 这是天然的接缝，同时解决两边短板

硬约束两路线公平对比：投影 vs 梯度引导，同一 benchmark 上跑一遍就有结论，实验成本可控，结论无论正负都有价值

④ 更多方向（P1）

预训练表征当生成模型 backbone：DeepLayout 训出了通用表征，但 FlowPlace / MacroDiff+ 的去噪网络都从零设计 GATv2+Attention

把「学权重调度」搬到布局阶段：Accelerating Detailed Routing 已在布线侧验证方法论，布局阶段还停留在贝叶斯优化 + 手工代理目标

建多任务后端评测基准：四套 benchmark、三类不可通约指标，基准本身就是贡献